

TalentFlow — System Architecture

React/Vite → Django REST Framework → PostgreSQL • v1.0

Architecture at a glance

Browser / React + Vite → Django REST Framework API → Django ORM → PostgreSQL

Frontend

React, JavaScript, Vite, Lucide icons and modern CSS provide the authenticated workspace. The UI calls versioned REST endpoints and stores the authentication token locally for the current demo workflow.

Backend

Django and Django REST Framework provide routing, serializers, authentication, business logic and database access. API endpoints are grouped under /api/v1/ for health, authentication, accounts, companies, jobs and applications.

Data layer

PostgreSQL stores users, companies, jobs, skills, applications and recruitment workflow data. Django relationships model recruiter/company ownership, job skills and application status history.

Core request flow

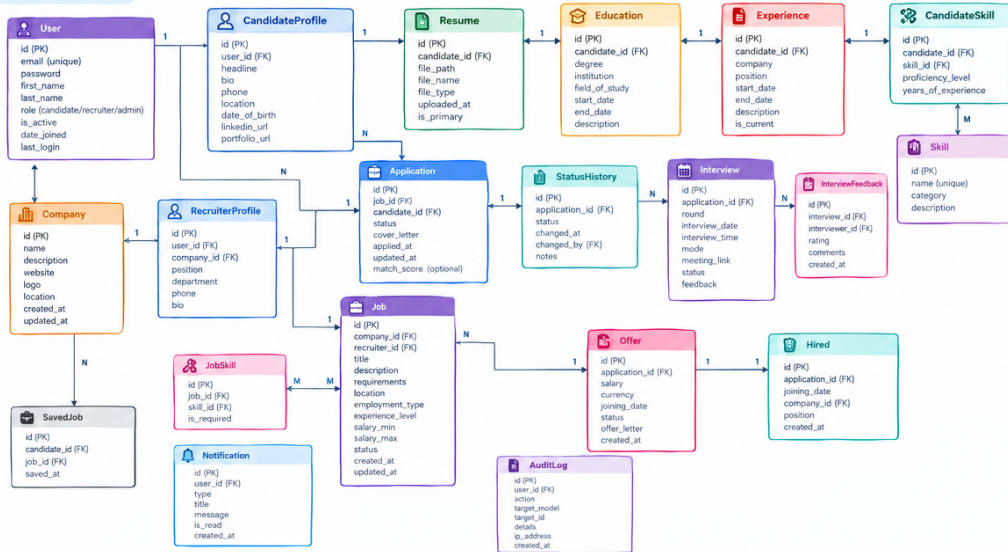
1	React UI	User performs an action such as creating a job, applying, or changing a pipeline status.
2	REST request	The frontend sends JSON to the Django API with Token authentication.
3	DRF validation	Serializers and view logic validate the request and enforce role-aware behavior.
4	Django ORM	The application reads or writes relational records using Django models.
5	PostgreSQL	Persistent data is stored and returned to the API.
6	UI refresh	React reloads the relevant data and updates the visible dashboard/list/pipeline.

Production direction

The blueprint also defines a broader production architecture with Docker, Nginx/Gunicorn, optional Redis/Celery background processing, file storage, testing, CI/CD and deployment. These are documented as future/progressive capabilities rather than claiming they are all active in the current MVP.

Database architecture reference

1. ER Diagram



2. Relationship Types

- One to One (1 — 1)
- One to Many (1 — N)
- Many to Many (M — N)

3. Entity Categories

- User & Profiles
- Recruitment & Jobs
- Applications & Pipeline
- Interviews & Feedback
- Offers & Hiring
- Company & Recruiter
- Skills & Matching
- Notifications & Logs

4. Key Relationships

- User 1 — 1 CandidateProfile / RecruiterProfile
- Company 1 — N Job
- Recruiter 1 — N Job
- Job 1 — N Application
- Candidate 1 — N Application
- Application 1 — N StatusHistory
- Application 1 — 1 Interview (or N)
- Interview 1 — N InterviewFeedback
- Job M — N Skill (via JobSkill)
- Candidate M — N Skill (via CandidateSkill)
- Application 1 — 1 Offer
- Offer 1 — 1 Hired (optional)
- Candidate 1 — N SavedJob
- User 1 — N Notification
- User 1 — N AuditLog

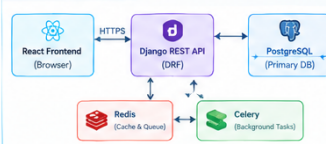
5. Database Schema Summary

Total Tables	22
Core Tables	14
Supporting Tables	8

6. Database Design Principles

- Use PostgreSQL for relational data
- Use proper indexing (FKs, frequently queried fields)
- Normalize to 3NF (avoid data redundancy)
- Use JSON fields for flexible data (e.g., skills, metadata)
- Implement soft delete (is_active / deleted_at)
- Use constraints (unique, not null, check)

7. Database Architecture



8. Additional Infrastructure

- Nginx** (Reverse Proxy)
- Gunicorn** (WSGI Server)
- Docker** (Containerization)
- Docker Compose** (Local Dev)
- CI/CD** (GitHub Actions)
- Monitoring & Logs** (Prometheus + Grafana)

9. What Goes in PostgreSQL vs Not

- Store in PostgreSQL (Database Tables)**
 - All core application data (users, jobs, applications, etc.)
 - Structured data (profiles, skills, status history, etc.)
 - Metadata and audit logs
 - Relationships and foreign keys
- Do NOT Store in PostgreSQL**
 - Frontend assets (in build/hosting)
 - Large files (resumes, logos) → use file storage (S3)
 - Cache data → use Redis
 - Background task queue → use Celery + Redis